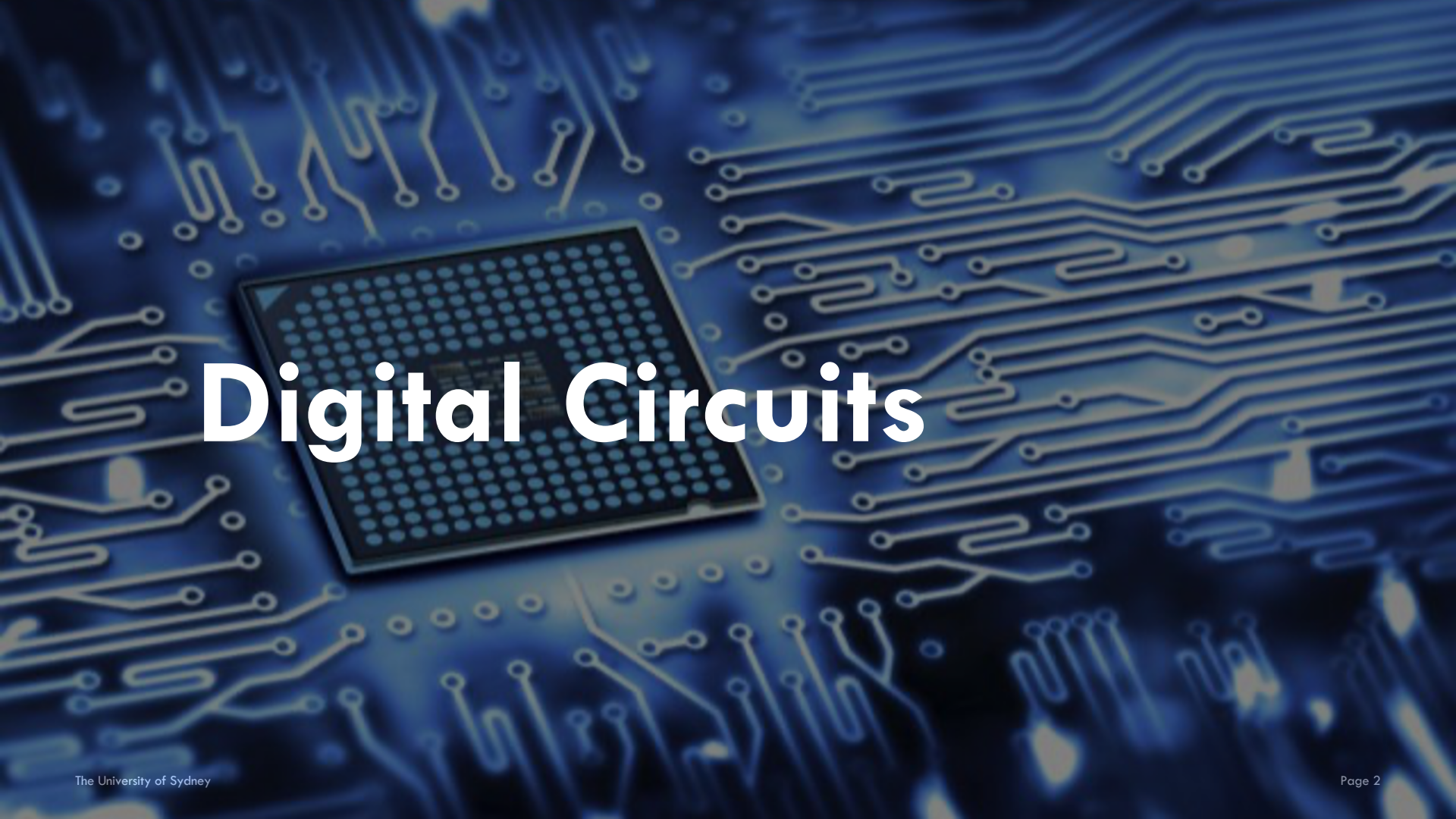
Mtrx 3700 : Mechatronics 3
Introduction to Digital Circuits and FPGA

Dr. Viorela ILA



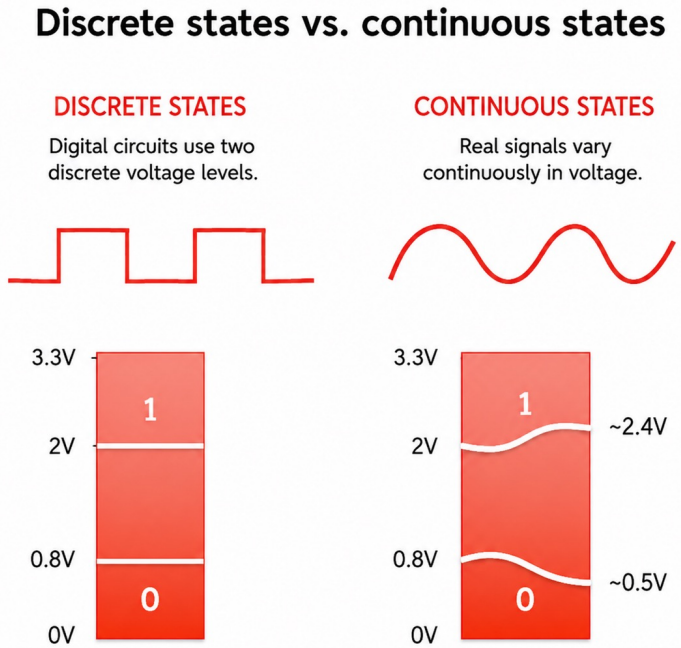
Digital Circuits

Digital Circuits

What means for something to be digital?

In the context of electronics “digital” refers to a circuit that deals with **discrete states** instead of a fully continuous set of values used in an analog system.

- Digital circuits are just analog circuits that we expect to behave a certain way. This abstraction is 0 and 1.
- For a circuit running on 3.3 V a zero is often represented from anything from 0 to 0.8 volts while a 1 is anything from 2 to 3.3 V.
- Logic gates accept a range of voltages for 0 and 1, but their outputs are driven closer to the ideal levels (0 V and 3.3 V), restoring the digital signal at each stage.
- The values between these two ranges are often referred to as **no-man's land** and the circuits are designed to avoid this region.



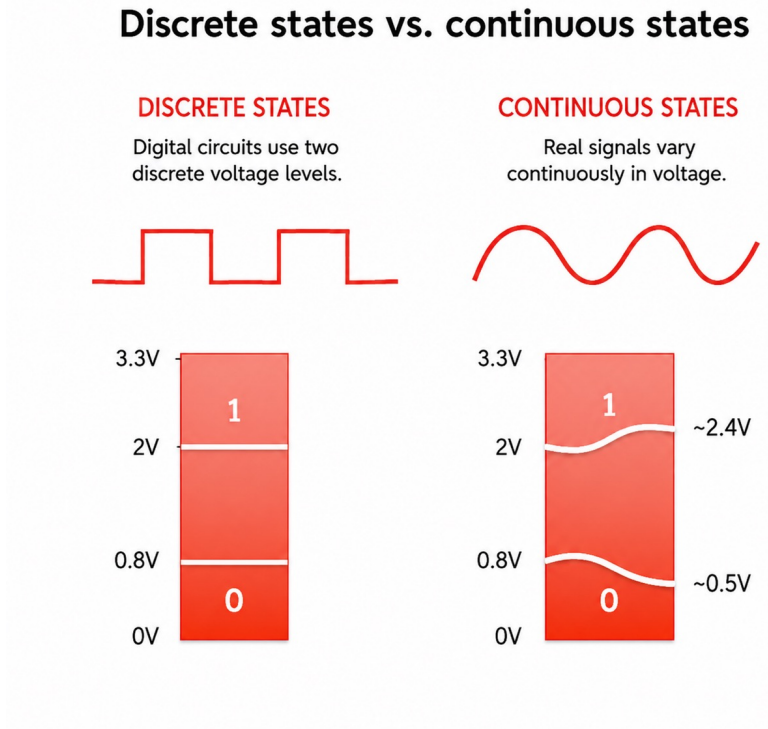
Why Digital Circuits

- The biggest reason that digital circuits are so widely used is because they are very **robust.**

- **Reliability**: Digital signals are more resistant to noise, it is easier to tell the difference between a 0 and a 1 than to precisely measure a varying analog signal.

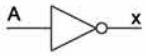
- **Storage & Processing**: Computers and memory store everything (numbers, text, images) as sequences of 0s and 1s.

- **Scalability**: Digital designs can be easily scaled and replicated using logic gates and programmable devices.



Logic Gates

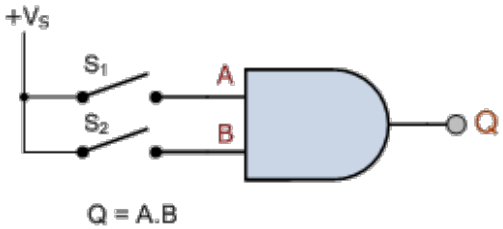
Logic gates are the building block of every digital circuit.

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A+B$	$\overline{A+B}$	$A \oplus B$	$\overline{A \oplus B}$
Symbol							

Truth Table																								
	A	X	B	A	X	B	A	X	B	A	X	B	A	X	B	A	X	B	A	X	B	A	X
	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	1
	1	0	0	1	0	0	1	1	0	1	1	0	1	0	0	1	1	0	1	1	0	1	0
			1	0	0	1	0	1	1	0	1	1	0	0	1	0	1	1	0	1	1	0	0
			1	1	1	1	1	0	1	1	1	1	1	0	1	1	0	1	1	0	1	1	1

We need to forget about all the messy real-world issues and live in the perfect world of **zeros and ones.**

Logic Circuits



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

Circuit to turn on a light from two switches: turn the lights on if we are home and it is dark.

Here **'0' = OFF** and **'1' = ON**

As long as we are consistent in our design the most obvious meaning to give them is "true" and "false" one is typically true and zero is typically false.

In general **'0' = FALSE** and **'1' = TRUE**

In practice we could use any combination of bits to mean anything we want they're really just arbitrary values after all.

Boolean Algebra

Mathematical framework used to describe and manipulate the logic of binary variables.

Key operations (Engineering notation):

AND: $A \cdot \mathrm{B}$

OR: $A + B$

NOT: $\bar{A}$ (or A')

XOR: $A \bigoplus B = \bar{A} \cdot B + A \cdot \bar{B}$

Simplifying Boolean expressions can make digital logic circuits simpler, cheaper, and faster.

Boolean Algebra

Operation	Boolean Algebra	Engineering Notation	Programming Notation	Logic Gate
AND	A·B or AB	AB (dot often omitted)	A && B or A & B	AND
OR	A + B	A + B	A || B or A | B	OR
NOT	A' or NOT A	$\overline{A}$ or A'	!A or ~A	NOT
XOR	A $\oplus$ B or A XOR B	A $\oplus$ B	A ^ B	XOR
XNOR	(A $\oplus$ B)' or A XNOR B	A $\odot$ B or (A $\oplus$ B)'	!(A ^ B)	XNOR
NAND	(A·B)'	(AB)' or $\overline{(AB)}$	!(A && B)	NAND
NOR	(A + B)'	(A+B)' or $\overline{(A+B)}$	!(A || B)	NOR

Boolean Algebra Simplification

Important simplification laws:

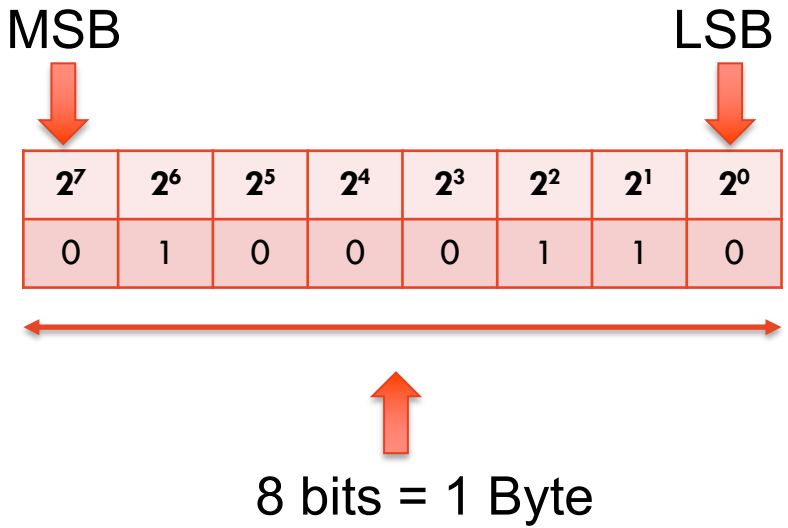
Domination Laws	Identity Laws	Idempotent Laws	Involution Law	Complement Laws
$X \cdot 0 = 0$ $X + 1 = 1$	$X \cdot 1 = X$ $X + 0 = X$	$X \cdot X = X$ $X + X = X$	$\bar{\bar{X}} = X$	$X \cdot \bar{X} = 0$ $X + \bar{X} = 1$
Absorption Laws			Distributive Laws	
$X + X \cdot Y = X$ $X \cdot (X + Y) = X$	$X + \bar{X} \cdot Y = X + Y$ $X \cdot (\bar{X} + Y) = X \cdot Y$		$X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$ $X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$	

De Morgan's Theorem: Allows conversion between AND and OR operations by negating the inputs and outputs. Very useful in simplifications!

$$\overline{X + Y} = \bar{X} \cdot \bar{Y}$$
$$\overline{X \cdot Y} = \bar{X} + \bar{Y}$$

Binary Numbers

$2^7 * 0 + \mathbf{2^6 * 1} + 2^5 * 0 + 2^4 * 0 + 2^3 * 0 + \mathbf{2^2 * 1} + \mathbf{2^1 * 1} + 2^0 * 0 = \mathbf{64 + 4 + 2 = 70}$



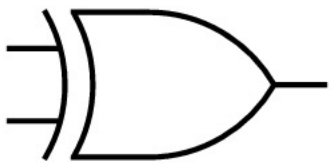
Binary Arithmetic

Adding two binary numbers

	0	1	0	0	0	1	1	0
+	0	1	0	0	0	1	0	1
	1	0	0	0	1	0	1	1

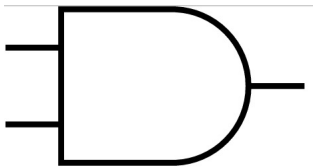
70
+ 69
139

B \ A	0	1
0	0	1
1	1	0



Result bit truth table

B \ A	0	1
0	0	0
1	0	1

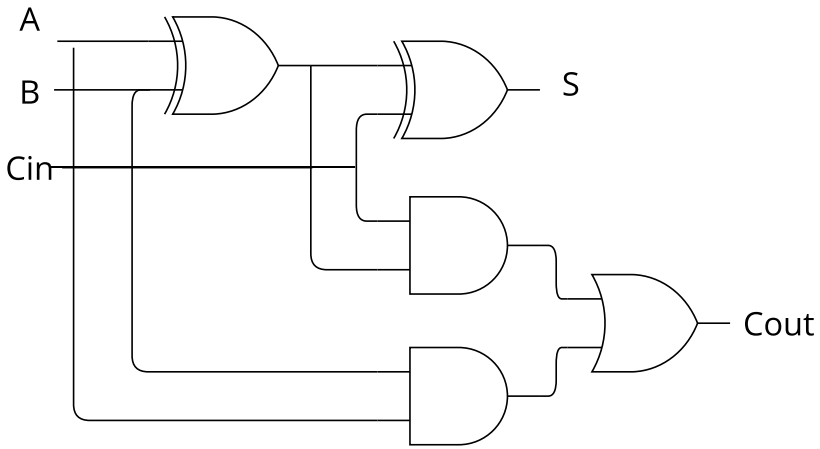


Cary bit truth table

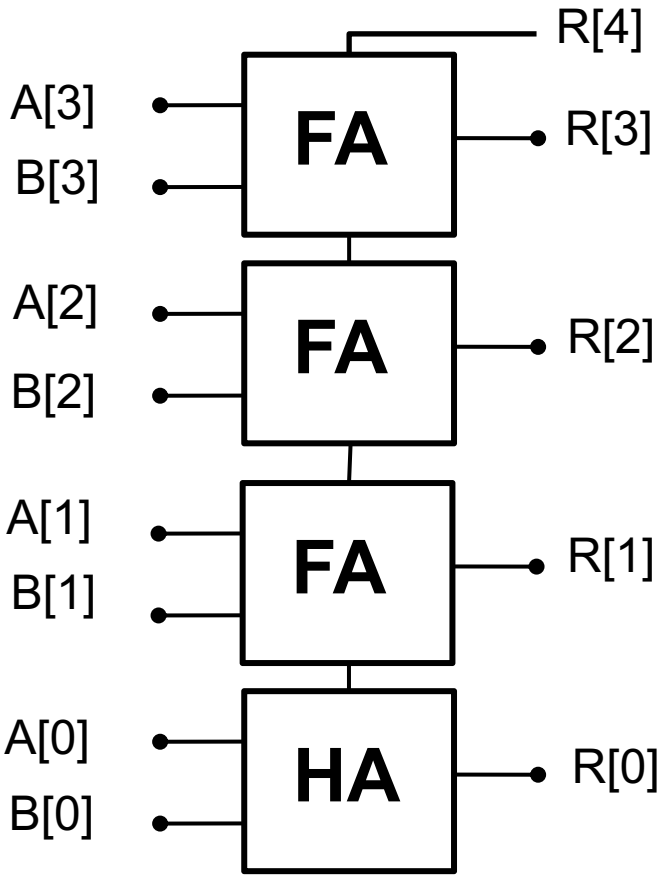
Full Adder

Inputs			Outputs	
A	B	C – IN	Sum	C - Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

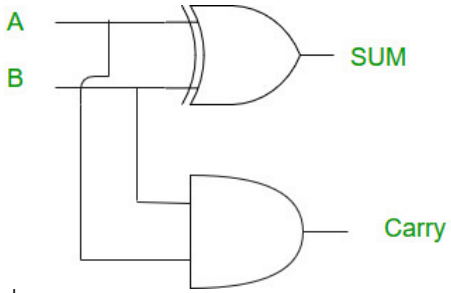
Full adder truth table



4-bit Adder



Half Adder

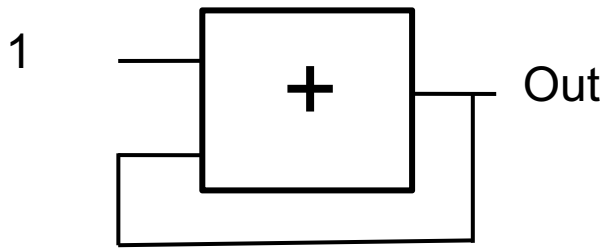


Simple Counter

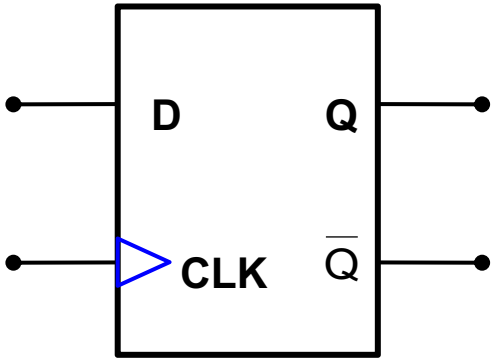
A counter is a circuit that just keeps adding one to the output.

When would this start counting?
How fast would it count?
How does it know when the output is ready?

We need something that has a **sense of time** and save the counter's values while the next value is being computed.



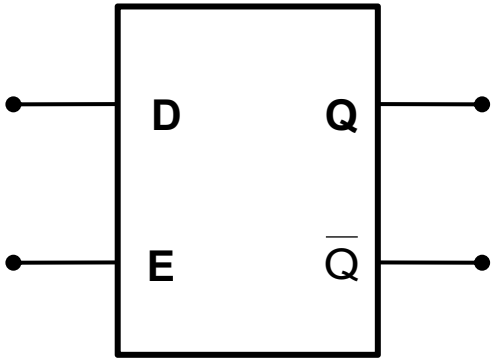
D Flip-Flop



A D (or Delay) **Flip Flop** is a digital electronic circuit used to delay the change of state of its output signal (Q) until the next rising edge of a clock timing input signal occurs.

D	CLK	Q(next)
0	No ↑	Q (Hold)
1	No ↑	Q (Hold)
0	↑	0
1	↑	1

D Latch



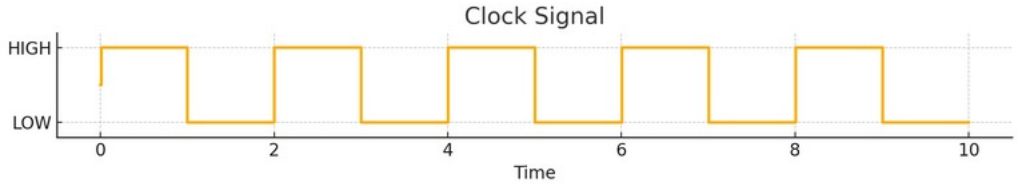
A D (or Delay) **Latch** is a digital electronic circuit that stores a single bit of data. When the enable (or clock) input is active, the output (Q) follows the input (D). When the enable is inactive, the output retains its previous state.

D	CLK (Enable)	Q(next)
0	0	Q (Hold)
1	0	Q (Hold)
0	1	0
1	1	1

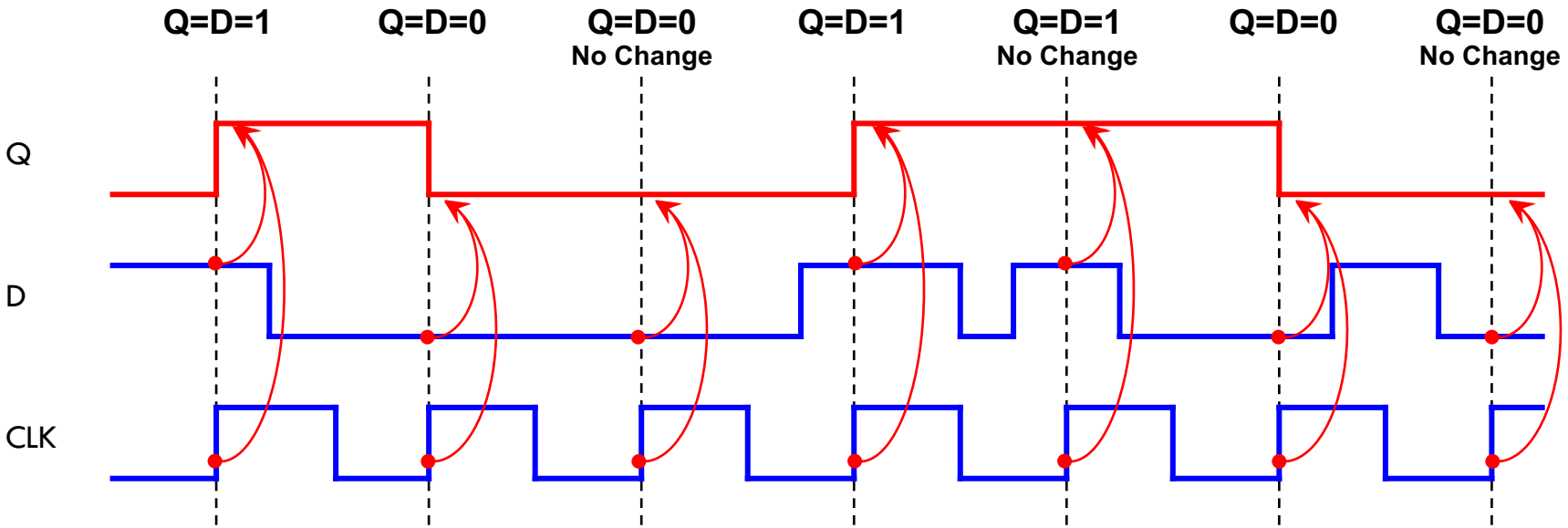
D Flip-Flop

- The D Flip Flop acts as an electronic memory component since the output remains constant unless deliberately changed by altering the state of the D input followed by a rising clock signal.
- The D Flip Flop is a building block of **sequential logic circuits.**

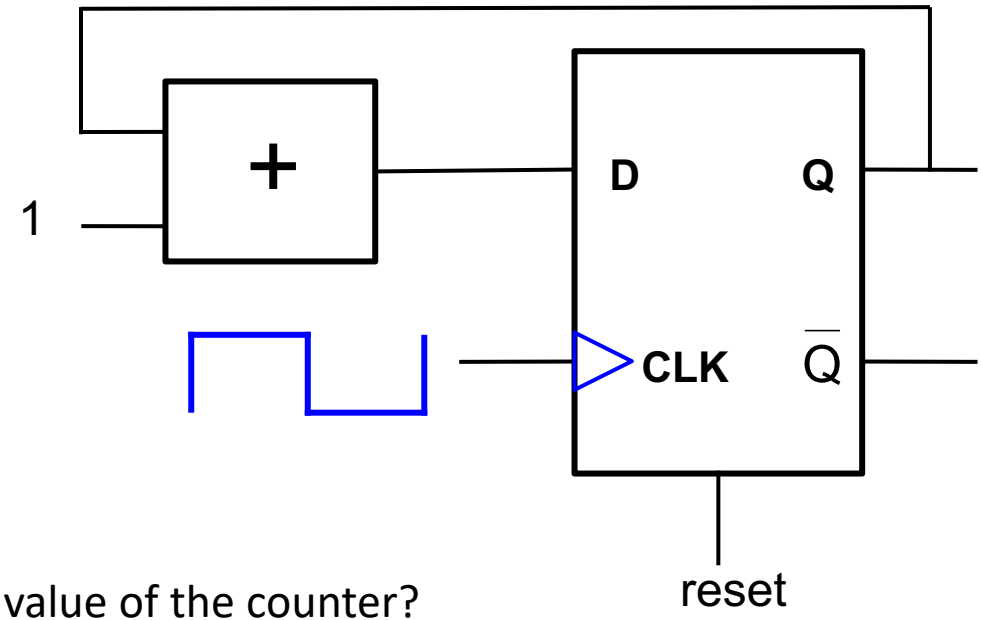
- If instead of the Clock signal we have an Enable input we have a D-Latch (level sensitive vs. edge sensitive).
- **Clocks** are a special type of signal used by digital circuits to control the flow of information.



D Flip-Flop


Q=D=1
Q=D=0
Q=D=0
No Change
Q=D=1
Q=D=1
No Change
Q=D=0
Q=D=0
No Change
Q
D
CLK

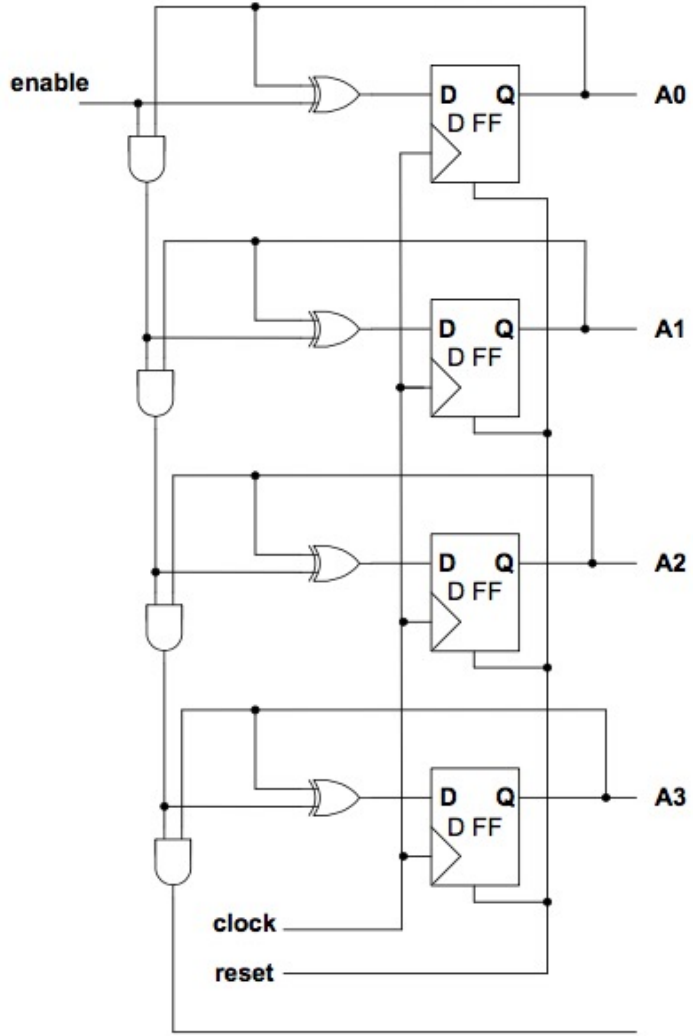
Counter



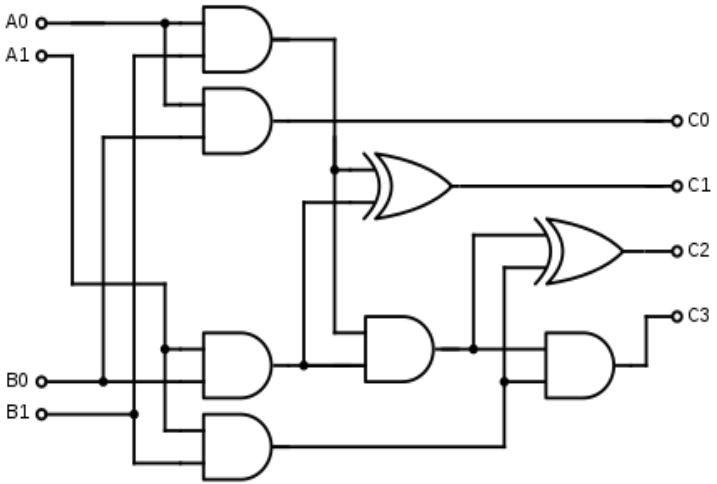
What is the initial value of the counter?
How to count up to more?

4-bits Counter

Design a counter that can increase up to a number represented by a certain number of bits.



Multiplier



A = A1A0
B = B1B0

INPUT A		INPUT B		OUTPUT			
A_1	A_0	B_1	B_0	C_3	C_2	C_1	C_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

Derive the logic equations for each bit of the product:

1.C0 = A0 & B0

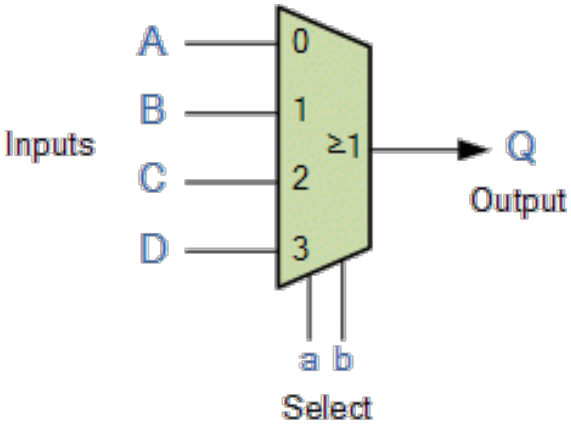
2.C1 = (A1 & B0) XOR (A0 & B1)

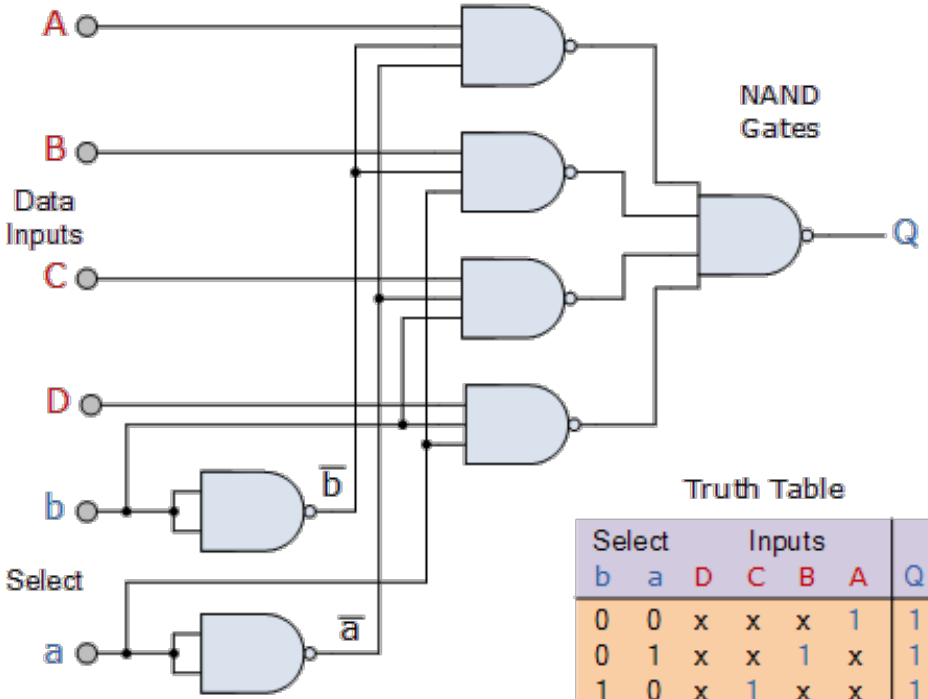
3.C2 = (A1 & B1) XOR (A0 & B1 & B0)

4.C3 = A1 & A0 & B1 & B0

Multiplexer

Output the value of data that corresponds to the value of the select input.



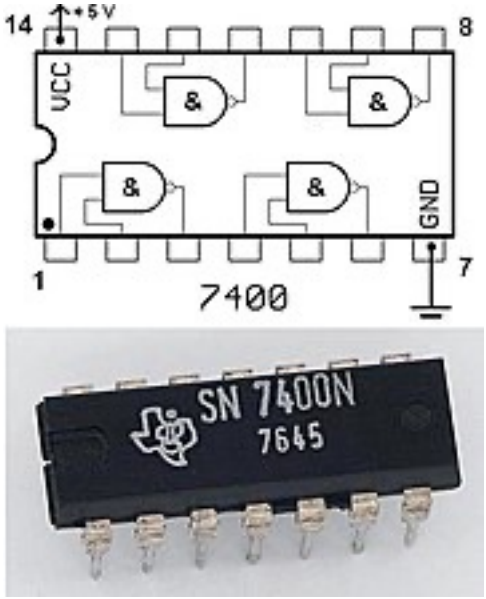


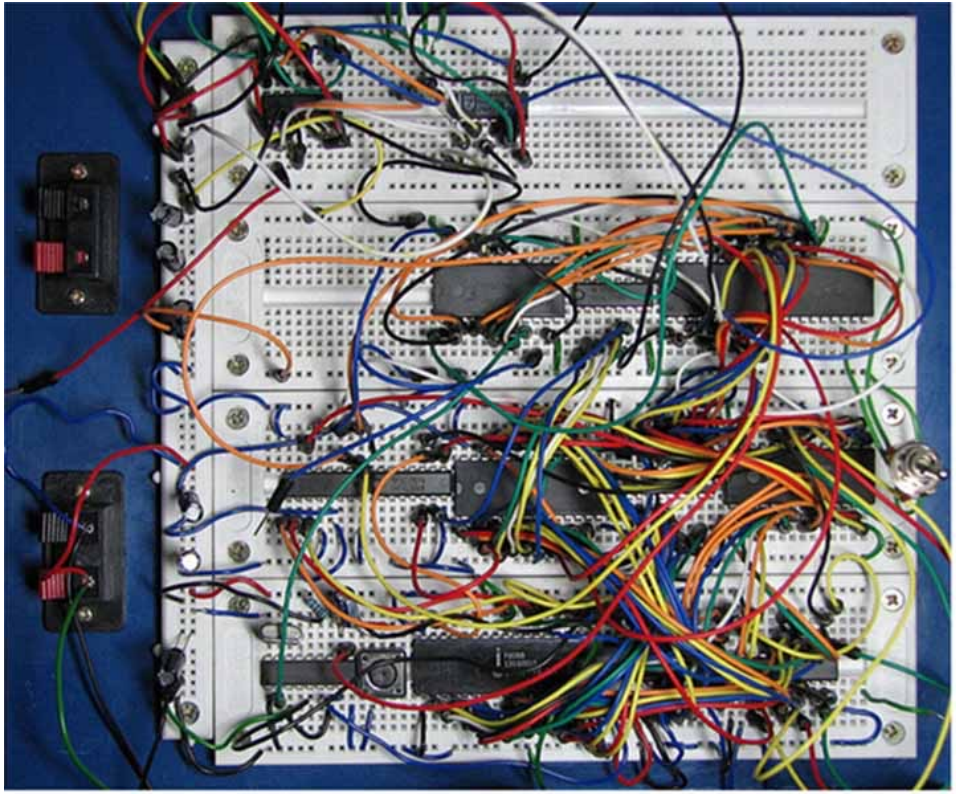
Truth Table

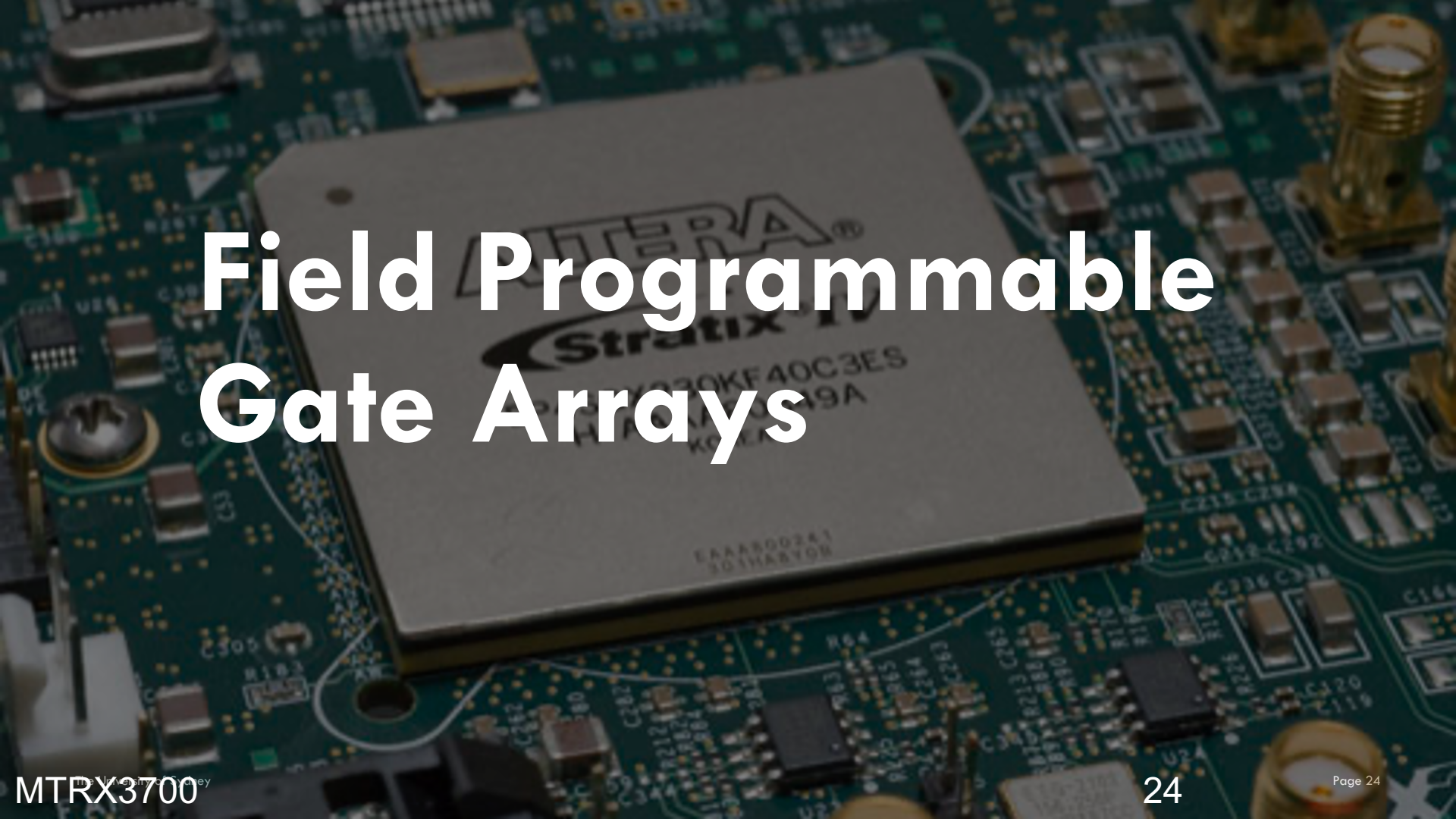
Select b	Select a	Inputs D	Inputs C	Inputs B	Inputs A	Q
0	0	x	x	x	1	1
0	1	x	x	1	x	1
1	0	x	1	x	x	1
1	1	1	x	x	x	1

4-to-1 Channel Multiplexer

Physical implementation

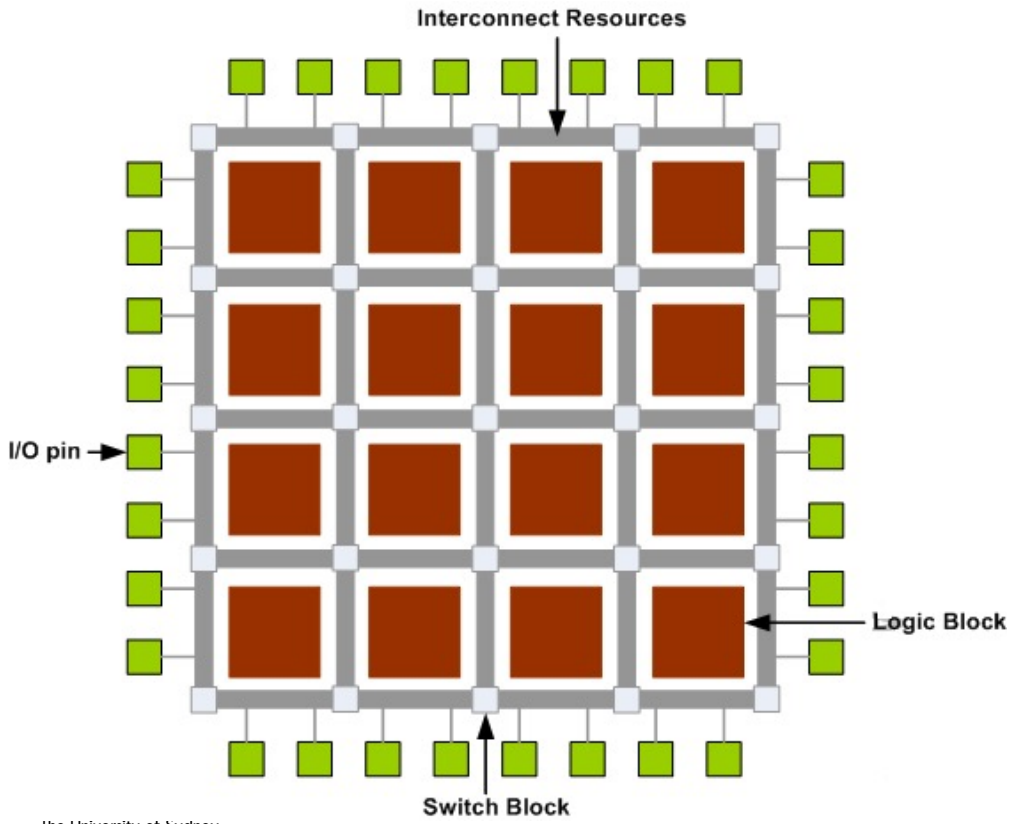






Field Programmable Gate Arrays

FPGA – General Structure



FPGAs are a type of chips that allow you to design a digital circuit and program them to become that circuit.

They can be reprogrammed as many times as you want and are relatively inexpensive.

FPGA configuration is specified using a hardware description language (HDL)

FPGA – Short History

- The FPGA industry evolved from the programmable read-only memory (PROMs) and other programmable logic devices (PLDs).

- In the 1980s, the Naval Service Warfare Center funded an experiment to develop a computer that would implement 600,000 reprogrammable gates.

- Altera was founded in 1983 and delivered the industry's first reprogrammable logic device in 1984.
- Xilinx co-founders invented the first commercially viable field programmable gate array in 1985, the XC2064.

- By the mid-90s, FPGA had exploded and revolutionized the industry.

Other Programmable Logic

- CPLD Complex Programmable Logic Device: devices with multiple PALs (Programmable Array Logic) in the same package with registered outputs and interconnecting programmable fabric. The main difference between CPLDs and FPGAs is that **CPLDs** Coarse-grained logic (based on *macrocells*) vs. Fine-grained logic (based on *Look-Up Tables*, or LUTs)

- Application-Specific Integrated Circuit (ASIC) and Application-Specific Standard Product (ASSP) are designed for a specific application while an FPGA is a multipurpose microchip you can reprogram for multiple applications.

FPGA – Manufacturers

1) AMD (Xilinx)
2) Intel (Altera)
3) Lattice Semiconductor
4) Achronix
5) QuickLogic Corporation
6) Microchip Technology
7) Microsemi Corporation
8) Efinix

Etc.











FPGAs

- Can offer **high performance**, often outperforming general-purpose processors (CPUs) in tasks like signal processing, cryptography, and data acceleration.

- For certain applications, FPGAs can be **more energy-efficient** than CPUs or GPUs because their hardware can be tailored precisely to the task at hand, reducing unnecessary computations.

- Excel in **parallel processing** tasks due to their architecture, allowing them to handle multiple operations simultaneously. This capability is crucial for applications like real-time data analysis and AI inference.

- They are used in a variety of fields including telecommunications, automotive, aerospace, and increasingly in data centers for accelerating specific workloads.

FPGA Applications

Avionics
IoT
Automotive
HPC









FPGA Applications

Space
Military
Gamming









FPGA Applications

Vision



Augmented Reality



Drive Assist Systems



Drone Vision



Satellite Vision



Medical Vision



Machine Vision



Broadcast

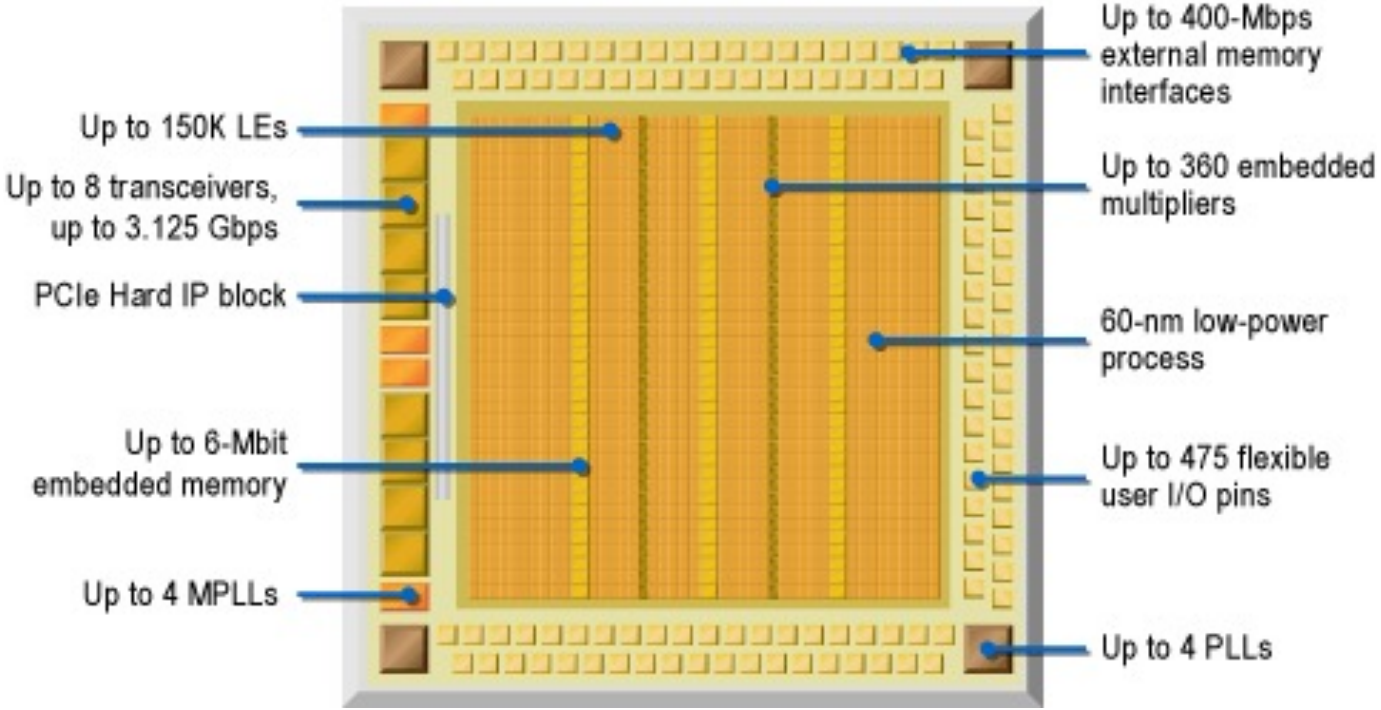


Physical Security

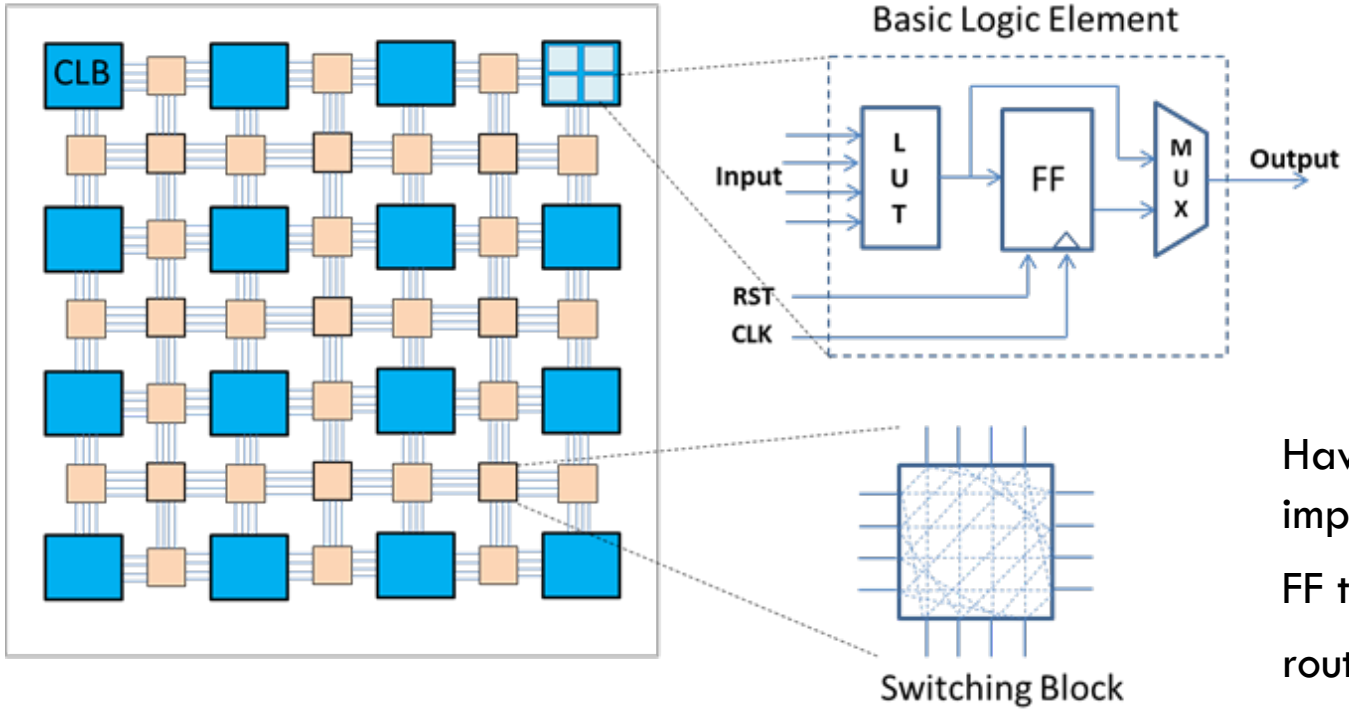


Thermal Vision

Altera Cyclone IV – Internal Stucture



Logic Elements (LE) – basic structure

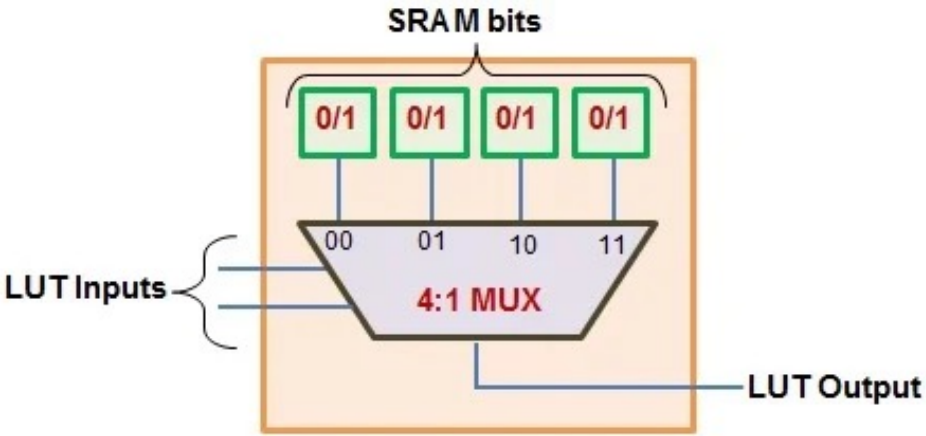


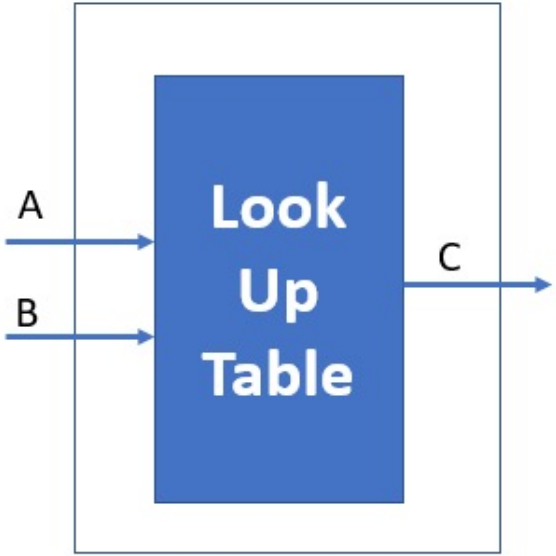
Have look-up tables that implement the logic,

FF that control the flow and

routing resources to connect everything up.

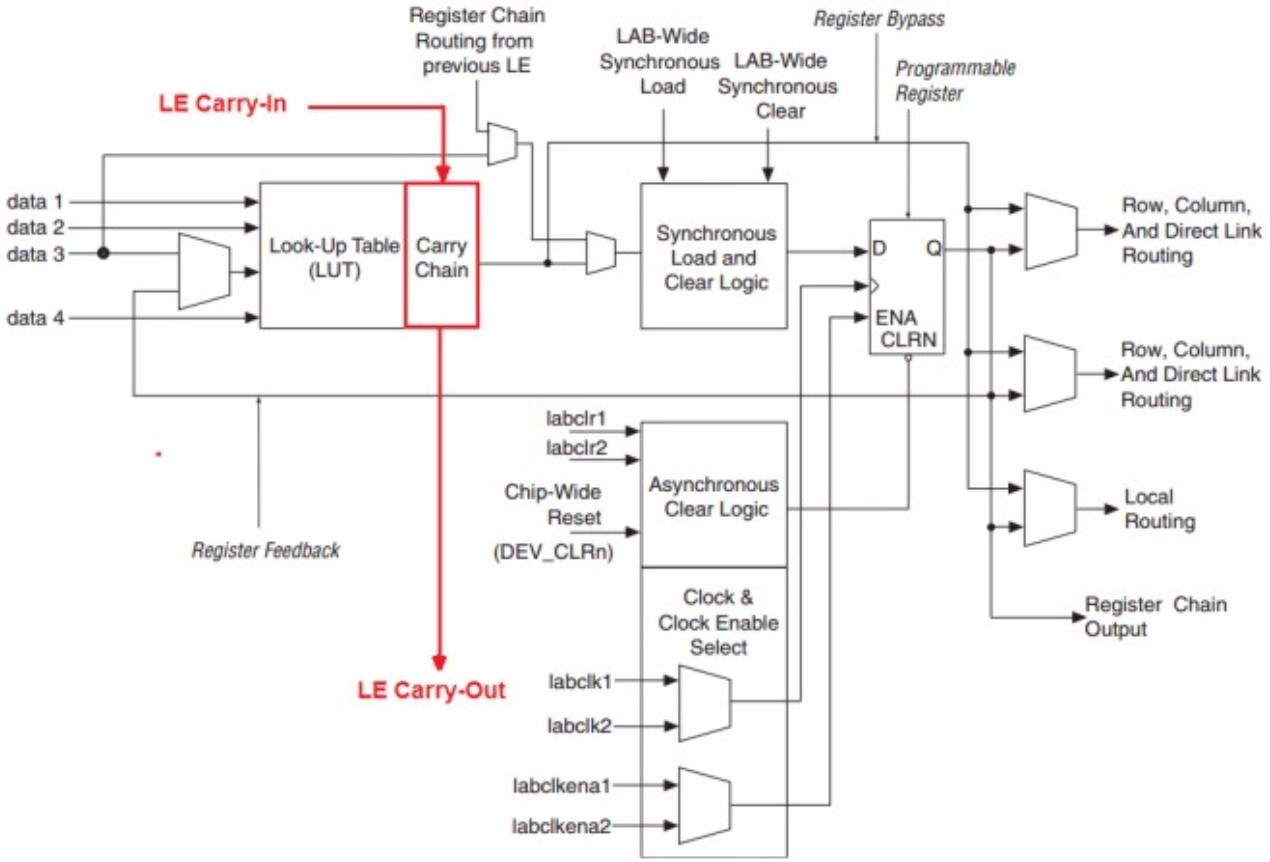
Look Up Table (LUT)



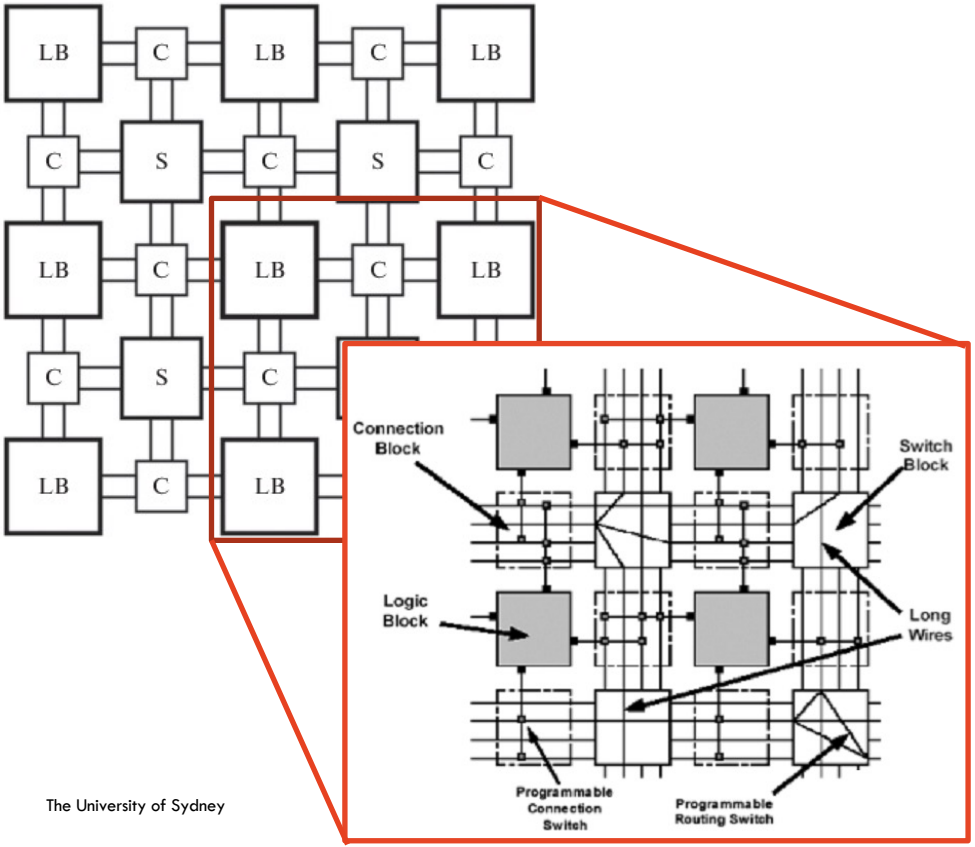


Input A	Input B	Output C
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

Logic Elements (LE) – basic structure



Routing in FPGA



The FPGA is made up of a sea of logic blocks.

The routing in a FPGA is hierarchical, at the lowest level, we have **routing switches** that are controlled by a SRAM, or FLASH, or a fuse.

There are many routing switches in local routing to connect various logic elements or blocks.

Common signals like clocks are routed on special routing networks called Global Networks.

Routing Switches

The type of configuration memory used to create routing switches, is an important aspect in determining differences in performance and capabilities in the FPGAs.

- SRAM FPGAs are programmable with highest density and lowest cost for equivalent logic. SRAM-based FPGAs require an external nonvolatile memory to hold their configuration pattern, which is transferred via serial link into the SRAM-based device.

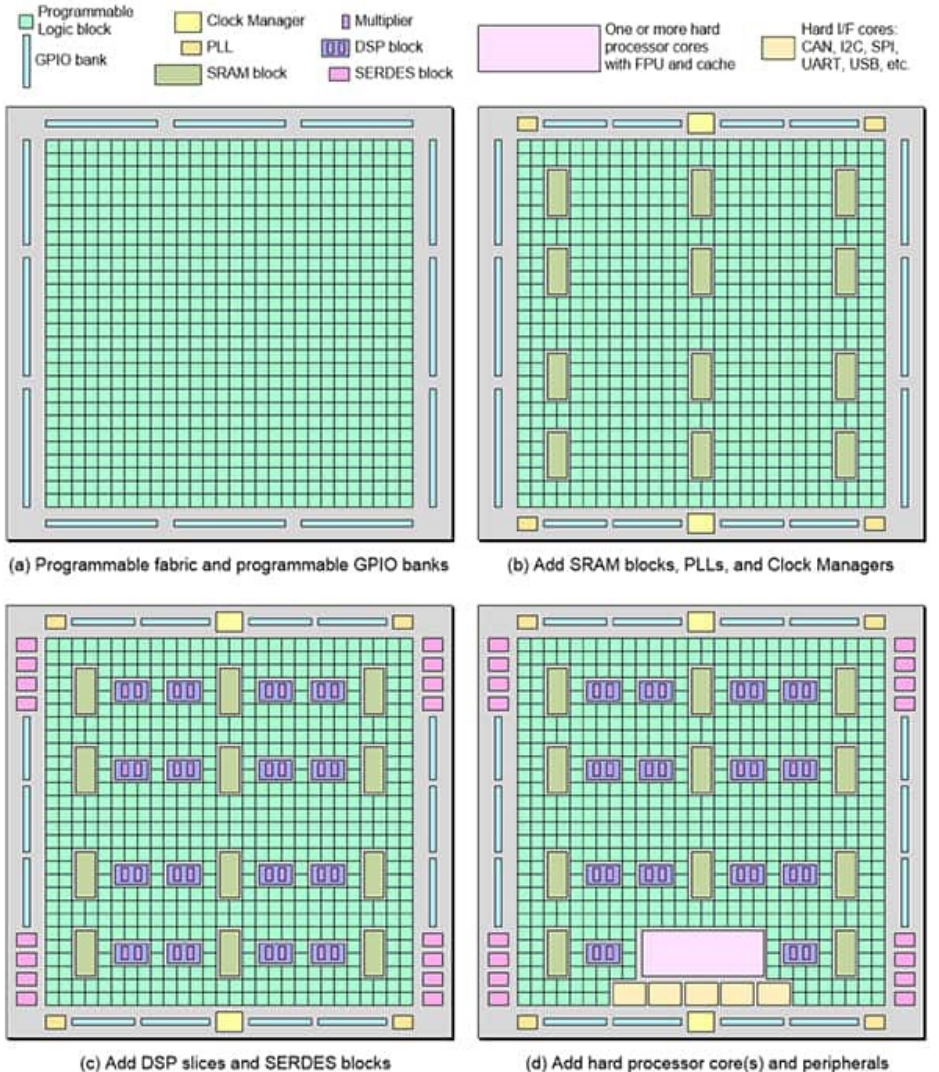
- FLASH FPGAs are also very reliable and there are also re-programmable, and eliminate the need of an external memory.

The FPGA vendors have a variety of offerings that include many configuration types.

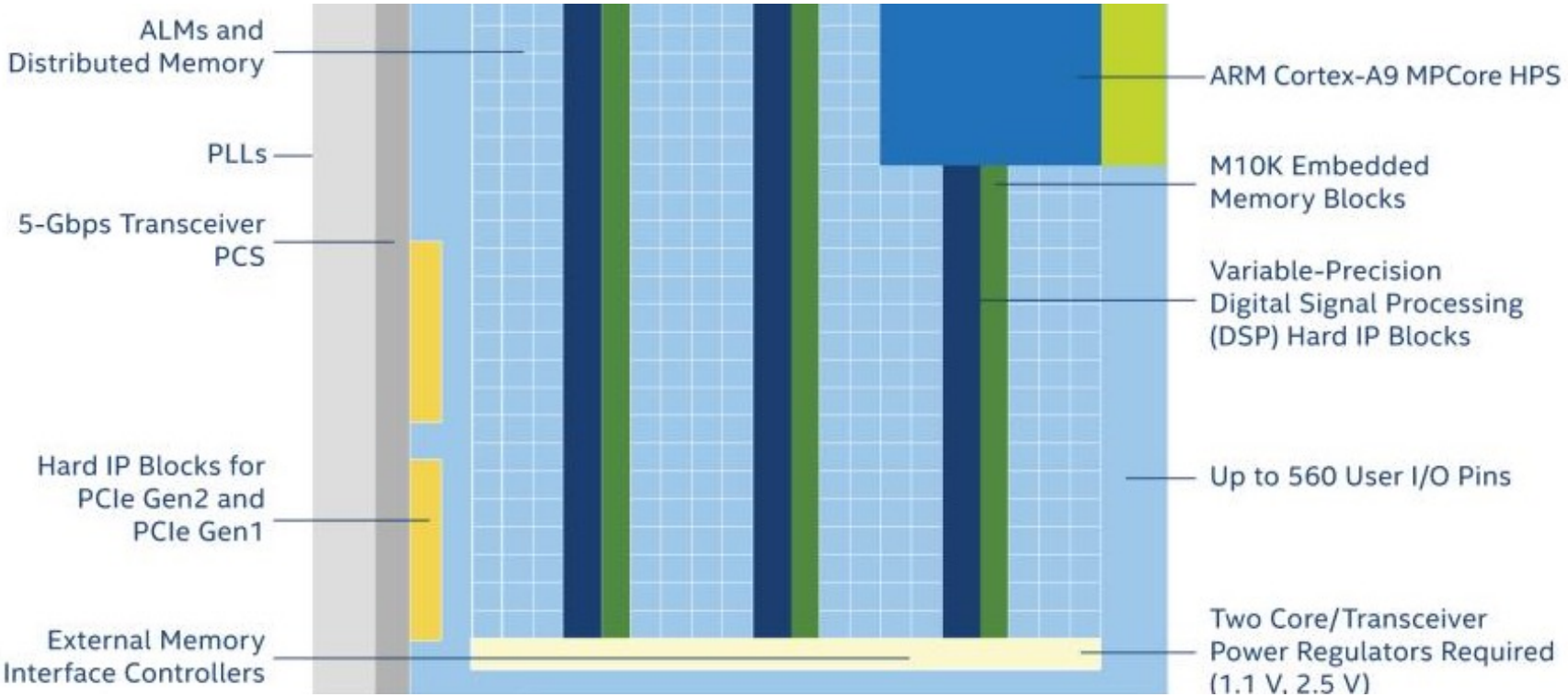
- Xilinx offers FLASH and SRAM FPGAs.
- Altera provides FLASH and SRAM FPGAs.
- Lattice sells SRAM FPGAs.

Evolution of FPGA

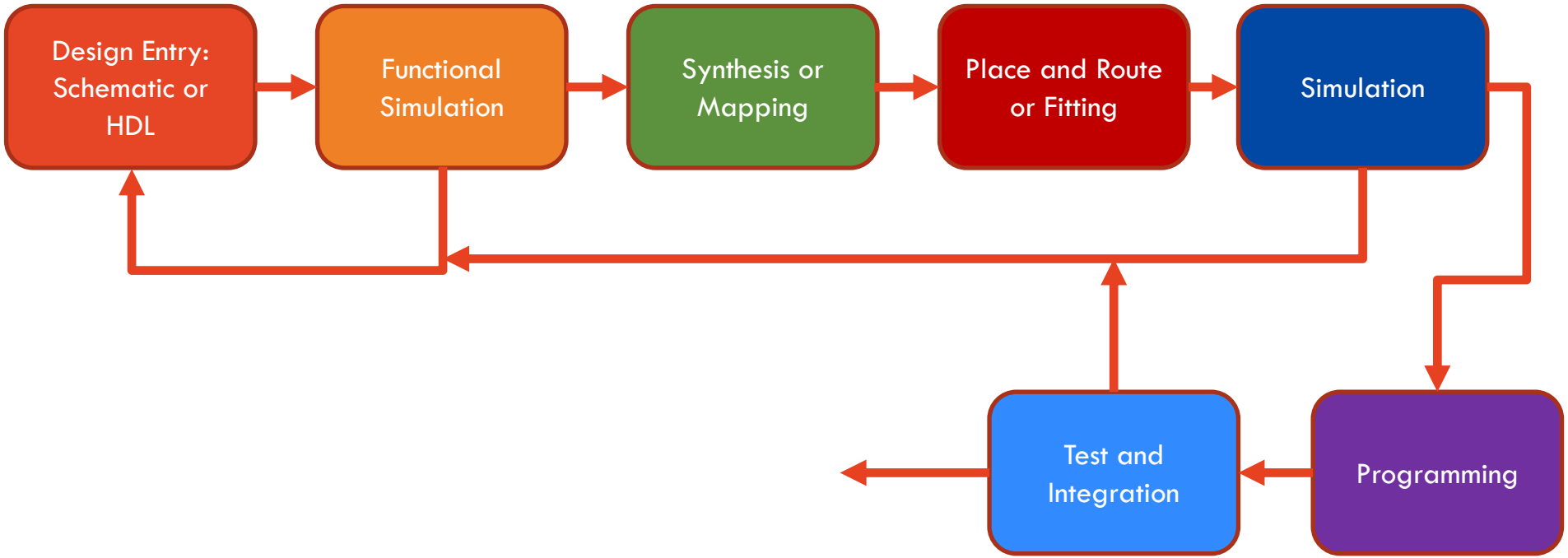
- Array of programmable logic blocks.
- **Blocks of SRAM**, phase lock loops (PLLs), and clock managers.
- **Digital signal processing** (DSP) blocks and high-speed serializer/deserializer (SERDES)
- **Peripheral** interface functions like CAN, I^2C, SPI, UART, and USB can be implemented as soft cores in the programmable fabric, but many FPGAs include them as hard cores in the silicon.
- **Microprocessors** can be implemented as soft cores in the programmable fabric or as hard cores in the silicon.



Altera Cyclone V SoC



FPGA Design


Design Entry: Schematic or HDL
Functional Simulation
Synthesis or Mapping
Place and Route or Fitting
Simulation
Programming
Test and Integration

FPGA Design

•**Design Entry** – the desired circuit is specified either by means of a schematic diagram, or by using a hardware description language (HDL), such as Verilog or VHDL .

•**Functional Simulation** – the circuit is tested to verify its functional correctness; this simulation does not take into account any timing issues

•**Synthesis** – the entered design is synthesized into a circuit that consists of the logic elements (LEs) provided in the FPGA chip. Will convert the HDL code to a netlist file.

•**Fitting** – the CAD Fitter tool determines the placement of the LEs defined in the *netlist* into the LEs in an actual FPGA chip; it also chooses routing wires in the chip to make the required connections between specific LEs.

A **netlist** is a file format that describes the components, connectivity, and optionally, the placement and routing of an electronic circuit. In the context of FPGAs, there exist two main types of netlists: the **unrouted netlist**, and the **routed netlist**.

FPGA Design

•**Timing Analysis** – propagation delays along the various paths in the fitted circuit are analysed to provide an indication of the expected performance of the circuit

•**Timing Simulation** – the fitted circuit is tested to verify both its functional correctness and timing

•**Programming and Configuration** – the designed circuit is implemented in a physical FPGA chip by programming the configuration switches that configure the LEs and establish the required wiring connections

Intel® QUARTUS ® Prime

The **Intel® Quartus® Prime Design Software** includes everything you need to design for FPGAs, SoCs, CPLD from design entry and synthesis to optimization, verification, and simulation.

Lite

Cyclone® IV,V
Intel® Cyclone® 10 LP
Max® Series

Standard

Intel® Arria 10
Arria® II, V
Cyclone® IV,V
Intel® Cyclone® 10 LP
Max® Series
Stratix® II, V

Pro

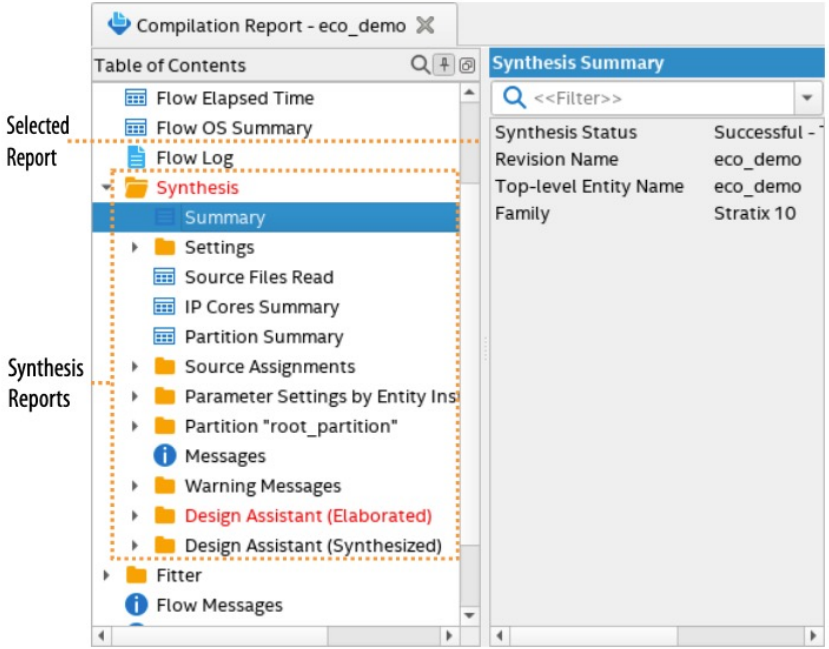
Intel® Arria 10
Intel® Cyclone® 10 GX
Intel® Agilex®
Intel® Stratix® 10

https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime.html

Design Entry

- Schematic entry: file:///Users/iviorela/Downloads/Quartus_Introduction_Schem.pdf
- Import of intellectual property (IP) blocks: https://www.intel.com/content/www/us/en/support/programmable/support-resources/intellectual-property/intellectual-property.html
- HDL text entry including VHDL, Verilog, and System-Verilog
- State machine entry: https://www.intel.com/content/www/us/en/docs/programmable/683230/18-1/state-machine-viewer.html
- Import of EDIF files (vendor neutral netlist files)

Design Analysis and Synthesis



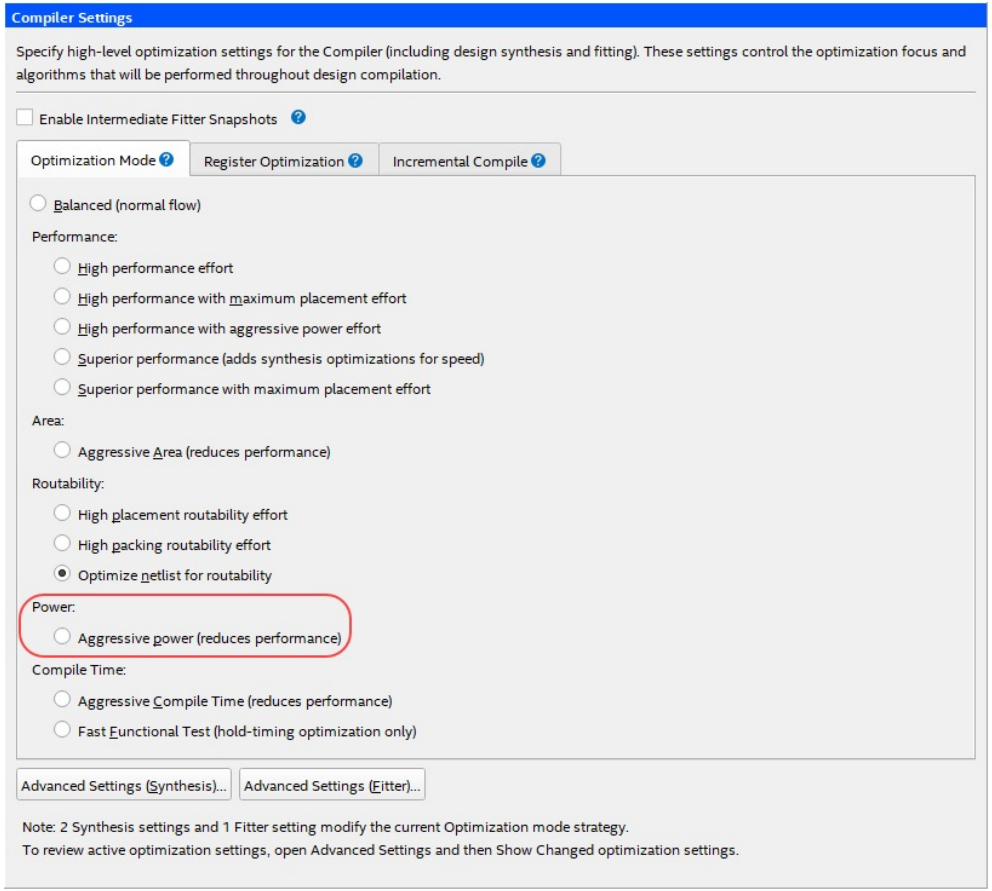
Quartus Prime's analysis and synthesis module:

- checks the design source files for errors,
- builds a database that puts all the design files in a hierarchy
- synthesizes and optimizes the logic design, and maps the design logic to device resources.

Fitting the design

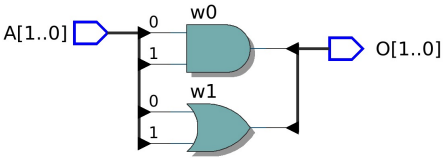
Fitting the design into the smallest possible part is a main challenge in FPGA design.

Quartus Prime allows control of the fitting process by choice of fitting approaches, including balanced, high performance for speed, low power, and small area.

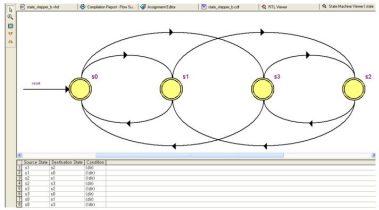


Views of the design in Quartus

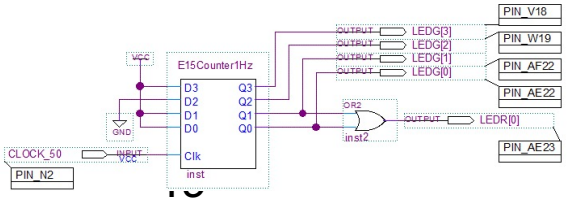
- Register Transfer Level (RTL) :
  - A design abstraction used to define digital circuits as a flow of signals between the elements
  - These diagrams offers a high-level representation of the circuits



- State Machine Viewer
  - The State Machine Viewer displays a graphical representation of the state machines in your design.

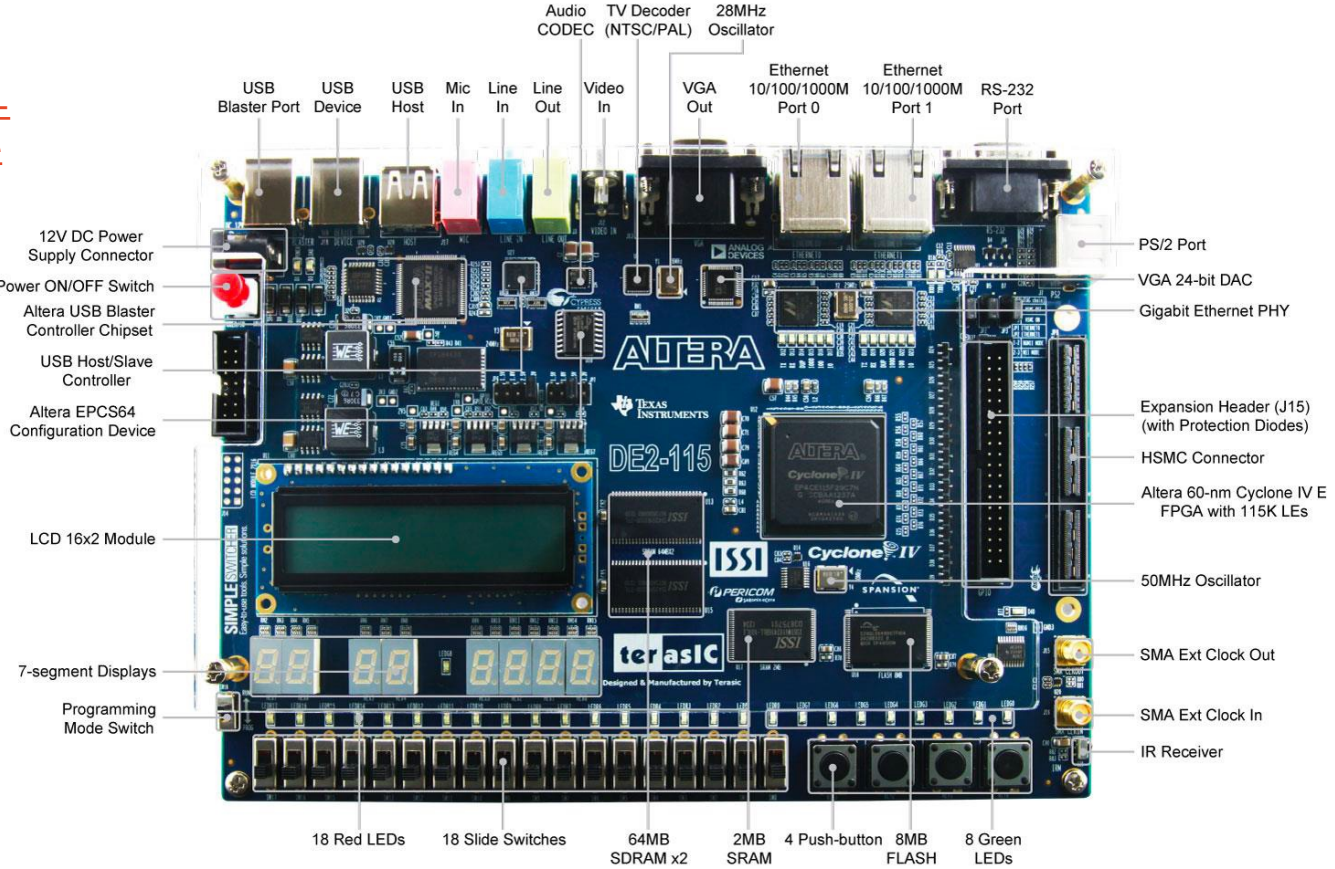


- Technology Map Viewer
  - Provides a technology-specific, graphical representation of FPGA designs after Analysis and Synthesis or after the Fitter maps the design into the target device
- Netlist Viewer
  - A graphical user-interface for viewing and manipulating nodes and nets in the netlist.
- Schematic Viewer
  - Contains a schematic representing the design logic in the netlist.



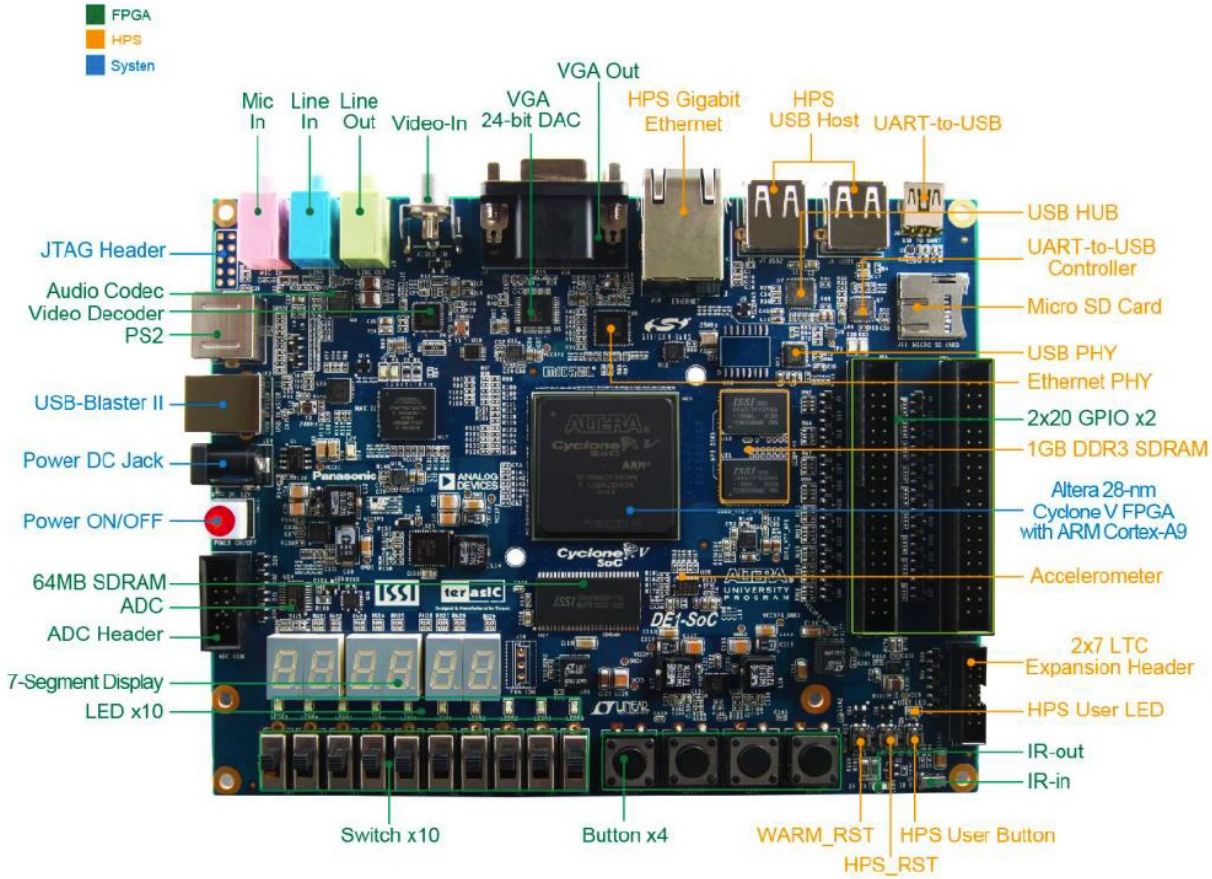
Hardware- DE2-115 board

https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=502



DE1-SoC

https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=167&No=836



Conclusions

- We learned about different levels of abstracting digital circuits
- We learned about main building blocks of the FPGA devices
- We highlighted the main components of an FPGA design
- We discuss Quartus Prime software